🚀 Collections Framework Masterclass

Chapter 20 — EnumMap — Complete Deep Dive

> **Interview Importance:** ⭐⭐⭐⭐☆

`EnumMap` is a specialized `Map` implementation designed specifically for **enum keys**.

The core idea:

> **When your keys are enum constants, EnumMap uses the enum's fixed set of constants to implement a compact, fast map—internally backed by an array.**

Think:

```text id="e1m7k4"
EnumMap ↓
Enum keys ↓
Array-based storage ↓
Fast + memory efficient
```

````
This is one of the best examples of choosing a collection based on the **nature of your key type**, rather than simply defaulting to `HashMap`.

---

# 1. What Is EnumMap?

Package:

```java id="x8q3m5"
java.util.EnumMap
````

It implements:

```text id="r6v2n9"
Map<K,V>
```

````
where:

```text id="c4m7p1"
K must be an enum type
````

Example:

```java id="a9k5w2"
enum Day { MONDAY, TUESDAY, WEDNESDAY }
```

EnumMap<Day, String> map = new EnumMap<>(Day.class);

```
---

# 2. Why Does EnumMap Exist?

Suppose you have:

```java id="m3p8x1"
enum Status {
    NEW,
    PROCESSING,
    COMPLETED,
    FAILED
}
```

You want:

```text id="v7k2q4" Status → description

```
You could use:

```java id="h5n9r3"
HashMap<Status, String>
```

But Java provides a more specialized option:

```java id="z4m6p8" EnumMap<Status, String>

```
Because the key space is known and finite.

---

3. Why Is an Enum Special?

An enum has a fixed set of constants.

Example:

```java id="q8x2m5"
enum Priority {
    LOW,
    MEDIUM,
```
```

```
    HIGH,
    CRITICAL
}
```

There are exactly four possible enum constants.

Conceptually:

```text id="j5r9k3" LOW → ordinal 0 MEDIUM → ordinal 1 HIGH → ordinal 2 CRITICAL → ordinal 3

```
EnumMap can exploit this fixed structure.

---

4. Internal Structure

The most important implementation concept:

> **EnumMap is internally array-based.**

Conceptually:

```text id="p3m7x8"
Enum constant
     ↓
ordinal()
     ↓
array index
     ↓
value
```

For:

```text id="c6q1v4" MONDAY TUESDAY WEDNESDAY

```
you can think:

```text id="n8r5k2"
index 0 → MONDAY value
index 1 → TUESDAY value
index 2 → WEDNESDAY value
```

This avoids the normal hashing machinery of HashMap.

5. Why Array-Based?

Because the possible keys are already known.

For:

```java id="w4m9p2" enum Status { NEW, PROCESSING, COMPLETED }

```
Java knows the universe of possible keys:

```text id="k7x3q5"
0 → NEW
1 → PROCESSING
2 → COMPLETED
```

So instead of:

```text id="r2m8v6" hash ↓ bucket ↓ node

```
EnumMap can conceptually do:

```text id="s5p1n9"
enum
 ↓
ordinal
 ↓
array[index]
```

6. EnumMap Construction

Most common constructor:

```java id="d8q4m2" EnumMap<Status, String> map = new EnumMap<>(Status.class);

```
The class literal tells EnumMap:

> "This is the enum type used as the key."

```
# 7. Why Is `Status.class` Required?

Consider:

```java id="x7m3p9"
EnumMap<Status, String> map =
    new EnumMap<>(Status.class);
```

EnumMap needs to know the key enum type so it can determine:

```text id="v5k8r2"
all enum constants

and build the appropriate internal structure.

---

# 8. Copy Constructor

You can also construct an EnumMap from another EnumMap:

```java id="n4q7m1"
EnumMap<Status, String> map2 =
    new EnumMap<>(map1);
```

This preserves the enum-key type and mappings.

---

9. Map Constructor

You can also construct one from a Map:

```java id="p8m2x6"
Map<Status, String> source = new HashMap<>();

EnumMap<Status, String> map = new EnumMap<>(source);
```

```
The source must contain enough information for EnumMap to determine the enum key
type, such as at least one mapping.

An empty generic `Map` doesn't provide runtime generic type information because
of type erasure.

---
```

```
# 10. Type Safety

EnumMap is strongly type-safe.

If:

```java id="r6x3k9"
enum Status {
    NEW,
    DONE
}
```

then:

```java id="q1m8v5" EnumMap<Status, String>
```

```
only accepts:

```text id="h7p4n2"
Status
```

as keys.

You cannot insert:

```java id="j5x9m3" "NEW"
```

```
as a key.

---

# 11. Basic Example

```java id="c8r2m7"
enum Status {
    NEW,
    PROCESSING,
    COMPLETED
}

EnumMap<Status, String> map =
    new EnumMap<>(Status.class);
```
```

```
map.put(Status.NEW, "Created");
map.put(Status.PROCESSING, "Running");
map.put(Status.COMPLETED, "Finished");
```

Now:

```java id="w4n7p1" System.out.println( map.get(Status.PROCESSING) );

```
Output:

```text id="m9x3q6"
Running
```

---

12. Ordering

EnumMap maintains entries in:

> **Natural order of the enum constants.**

That means the order in which constants are declared.

Example:

```java id="v2k8r4" enum Priority { LOW, MEDIUM, HIGH, CRITICAL }

```
EnumMap iteration follows:

```text id="p6m1x9"
LOW
MEDIUM
HIGH
CRITICAL
```

This is true even if you insert them in another order.

---

13. Insertion Order Does NOT Matter

```java id="q3r7m2" map.put(Priority.HIGH, "H"); map.put(Priority.LOW, "L"); map.put(Priority.CRITICAL, "C"); map.put(Priority.MEDIUM, "M");
```

````
Iteration is still:

```text id="n8x5k1"
LOW
MEDIUM
HIGH
CRITICAL
````

because EnumMap follows enum declaration order.

---

14. EnumMap vs LinkedHashMap

This distinction is important.

LinkedHashMap

```text id="j6m2p8" Insertion/access order

````
EnumMap

```text id="x9q4r1"
Enum declaration order
````

So:

```text id="f3k7m5" EnumMap ≠ insertion-order Map

````
---

15. EnumMap vs TreeMap

TreeMap sorts using:

```text id="a8m3q7"
Comparable / Comparator
````

EnumMap follows:

```text id="r5x9n2" enum declaration order

````
EnumMap is specialized specifically for enum keys and is generally much more
compact.

---

16. `ordinal()`

Every enum constant has an ordinal:

```java id="p7k4m8"
Status.NEW.ordinal();
````

If:

```java id="v2r6x9" enum Status { NEW, PROCESSING, COMPLETED }
```

````
then:

```text id="w5n8q3"
NEW        → 0
PROCESSING → 1
COMPLETED  → 2
````

EnumMap can exploit this fixed ordinal positioning internally.

---

17. Important Warning — Don't Persist Ordinals

You should **not** use:

```java id="q6m3x8" ordinal()
```

````
as a persistent business identifier.

For example, don't store:

```text id="n8p2r5"
0 → NEW
1 → PROCESSING
````

in a database and assume the values will never change.
````

If enum constants are reordered:

```java id="x4k9m1" enum Status { PROCESSING, NEW, COMPLETED }
```

```
the ordinals change.

Use an explicit field if you need a stable business code.

---

# 18. EnumMap Uses Enum Identity

Enum constants have special identity semantics.

For example:

```java id="m5r8q2"
Status.NEW == Status.NEW
```

is:

```text id="j7x3p9" true
```

```
Each enum constant is a singleton instance within its enum type.

This makes enum keys particularly suitable for specialized collection
implementations.

---

# 19. Null Keys

This is an important difference from HashMap.

EnumMap does **not permit null keys**.

```java id="c8m4x7"
map.put(null, "Something");
```

throws:

```text id="q2n9p5" NullPointerException
```

````
Why?

Because every valid key must be one of the enum constants.

There is no `null` enum constant.

---

# 20. Null Values

Null values are allowed.

```java id="r6p3k8"
map.put(Status.NEW, null);
````

This is valid.

Therefore:

```text id="v5x1m9" null key → ❌ null value → ✅

````
---

21. Duplicate Keys

Like every Map:

```text id="j8m4q2"
one key → one mapping
````

Example:

```java id="p7x3n5" map.put(Status.NEW, "A"); map.put(Status.NEW, "B");

````
Final mapping:

```text id="k2r9m6"
NEW → B
````

````

22. Duplicate Values

Allowed:

```java id="a5q8x3" map.put(Status.NEW, "Pending"); map.put(Status.PROCESSING, "Pending");

```
Both mappings can contain the same value.

---

23. Complexity

EnumMap provides very efficient basic operations.

Conceptually:

```text id="n7m3p8"
get()     → O(1)
put()     → O(1)
remove()  → O(1)
containsKey() → O(1)
```

Because the key maps directly to an array position.

This is one of EnumMap's major advantages.

---

24. Why Is EnumMap Faster Than HashMap?

Conceptually:

HashMap

```text id="k4r8m1" key ↓ hashCode() ↓ bucket calculation ↓ bucket lookup ↓ possibly compare keys

```
EnumMap

```text id="z6x2p9"
enum key
 ↓
ordinal / enum position
```

```
 ↓
array access
```

Fewer moving parts make EnumMap efficient.

Actual performance depends on workload and JVM details, but the structural advantage is clear.

---

25. Memory Efficiency

HashMap requires structures such as:

```text id="f3m7q8"
table
nodes
hash information
references
```

```
EnumMap can use an array indexed according to the enum's constants.

Therefore:

```text id="r9x2k5"
EnumMap
→ compact representation
```

especially when the enum type is relatively small.

---

26. Sparse vs Dense Enum Usage

Suppose:

```java id="p8m4q1"
enum State { A, B, C, D, E, F, G, H, I, J }
```

```
but you only store:

```text id="w5n9x2"
A
J
```

EnumMap's internal representation is still based on the enum universe.

This is an important trade-off:

EnumMap is optimized around a fixed finite enum key space, not around minimizing storage for only the currently present keys.

---

27. EnumMap vs HashMap

Feature	EnumMap	HashMap
Key type	Enum only	Any object
Internal structure	Array-based	Hash table
Average `get()`	O(1)	O(1)
Ordering	Enum declaration order	No guaranteed order
Null key	❌	One allowed
Null values	✅	✅
Memory efficiency	High for enum keys	General-purpose
Thread-safe	❌	❌
Specialized	✅	❌

---

28. EnumMap vs EnumSet

These two are often confused.

EnumSet

Stores:

```text id="m7x3p9"
only enum constants
```

```
Example:

```java id="q4r8n2"
EnumSet<Day>
```

Think:

```text id="s6k1v5"
"Which enum values are selected?"
```
```

```
### EnumMap

Stores:

```text id="j9p2m7"
enum → value
```

Think:

```text id="x5r8k3" "What value is associated with this enum?"
```

```
---

# 29. Example: EnumSet

```java id="w2m6q9"
EnumSet<Day> workingDays =
    EnumSet.of(
        Day.MONDAY,
        Day.TUESDAY
    );
```

Concept:

```text id="c4x7p1" MONDAY → present TUESDAY → present
```

```
No associated values.

---

# 30. Example: EnumMap

```java id="a8m3r6"
EnumMap<Day, String> schedule =
    new EnumMap<>(Day.class);

schedule.put(
    Day.MONDAY,
    "Development"
);

schedule.put(
    Day.TUESDAY,
```
```

```
    "Testing"
);
```

Concept:

```text id="n5q9x2" MONDAY → Development TUESDAY → Testing

```
---

31. EnumMap + State Machines

This is a very good real-world use case.

Suppose:

```java id="j7r4m8"
enum State {
    NEW,
    PROCESSING,
    COMPLETED,
    FAILED
}
```

You can map:

```text id="p3x8q1" State → allowed transitions

```
Example:

```java id="v6m2k9"
EnumMap<State, Set<State>> transitions =
    new EnumMap<>(State.class);
```

Conceptually:

```text id="f5r9x3" NEW → PROCESSING PROCESSING → COMPLETED / FAILED FAILED → ...

```
This creates a clean state-machine representation.

---

32. EnumMap for Configuration

```
Suppose:

```java id="b8m4q7"
enum Environment {
    DEV,
    TEST,
    PROD
}
```

You can store:

```java id="k2x9m5" EnumMap<Environment, String> urls = new EnumMap<>(Environment.class);
```

```
Then:

```text id="n7r3p8"
DEV  → development URL
TEST → testing URL
PROD → production URL
```

---

33. EnumMap for Strategy Selection

Suppose:

```java id="w4q8m2" enum PaymentType { CARD, UPI, NET_BANKING }
```

```
You could map:

```java id="p6x3r9"
EnumMap<PaymentType, PaymentProcessor>
```

Concept:

```text id="c8m5k1" CARD → CardProcessor UPI → UpiProcessor NET_BANKING → NetBankingProcessor
```

```
This avoids large `if-else` or `switch` blocks in some designs.

---
```
```

```
# 34. EnumMap for HTTP/Business Status

Example:

```java id="h3r7q9"
enum HttpStatusType {
    SUCCESS,
    CLIENT_ERROR,
    SERVER_ERROR
}
```

Map:

```java id="x8m2p5" EnumMap<HttpStatusType, Integer>
```

```
Could associate:

```text id="v4k9n1"
SUCCESS       → 200
CLIENT_ERROR  → 400
SERVER_ERROR  → 500
```

---

35. Iteration

EnumMap iteration follows enum declaration order.

```java id="j6q3m8" for (Map.Entry<Status, String> entry : map.entrySet()) {
```

```
System.out.println(entry);
```

} `If the enum is:` java id="p9x4r2" NEW, PROCESSING, COMPLETED ```

iteration follows:

```text id="m5k8q1" NEW PROCESSING COMPLETED
```

```
for entries that are present.

---
```

````
# 36. Missing Entries

Suppose:

```java id="q7m2x8"
enum Status {
    NEW,
    PROCESSING,
    COMPLETED
}
````

but only:

```java id="z4r9p3" map.put(Status.COMPLETED, "Done");
````

````
exists.

Iteration does **not** produce empty entries for:

```text id="f6k1m5"
NEW
PROCESSING
````

Only actual mappings are returned.

Conceptually:

```text id="n8x3q7" NEW → absent PROCESSING → absent COMPLETED → Done
````

````
---

# 37. `containsKey()`

Use:

```java id="k5m8r2"
map.containsKey(Status.NEW);
````

to determine whether the enum constant has an associated mapping.

This is different from:

```java id="p4x7n9" map.get(Status.NEW) == null

````
because a null value is allowed.

---

38. Null Value Trap

Because EnumMap allows null values:

```java id="c8q3m6"
map.put(Status.NEW, null);
````

then:

```java id="r2x9k5" map.get(Status.NEW)

````
returns:

```text id="v6m1p8"
null
````

But that doesn't tell you whether:

```text id="j4q7x3" key absent

````
or:

```text id="m8r2n5"
key present with null value
````

Use:

```java id="x5k9p1" containsKey()

````
to distinguish them.

---

39. EnumMap Is Not Thread-Safe

Like most standard Map implementations:
````

```text id="q3m7x8"
EnumMap
```

is not inherently thread-safe.

If multiple threads mutate it concurrently, use appropriate synchronization/concurrency mechanisms.

---

40. Synchronized Wrapper

If appropriate, you can use:

```java id="n6r2m9"
Map<Status, String> map = Collections.synchronizedMap( new EnumMap<>(Status.class) );
```

```
But remember:

> A synchronized wrapper is not the same as a purpose-built concurrent
collection.

---

# 41. Does EnumMap Use Hashing?

This is a useful interview question.

The answer is:

> **It doesn't use hashing in the same way HashMap does.**

Its internal representation is array-based and exploits the known enum key
universe.

Conceptually:

```text id="f7x3m8"
Enum
 ↓
known position
 ↓
array
```
```

rather than:

```text id="p2r9k5" hash ↓ bucket

```
---

42. Does EnumMap Use `ordinal()` Directly?

For interview purposes, think:

> **EnumMap can exploit the enum's ordinal/declared position to provide array-based storage.**

The exact implementation should not be reduced to "it literally does `values[key.ordinal()]` for everything." The JDK implementation has details such as masking null values and handling the enum universe.

The important architectural point is:

```text id="k8m4q2"
Enum universe
+
array storage
=
efficient Map
```

---

43. Why EnumMap Is Type-Safe at Runtime

Generics are erased at runtime.

Yet EnumMap needs to know:

```text id="r5x7n3" which enum class?

```
That's why:

```java id="j2m8q6"
new EnumMap<>(Status.class)
```

is useful.

It gives the implementation the actual enum class.

---

44. EnumMap and Enum Declaration Order

Suppose you change:

```java id="w3q9m5" enum Priority { LOW, MEDIUM, HIGH }
```

to:

```java id="a6k2r8"
enum Priority {
    HIGH,
    LOW,
    MEDIUM
}
```

EnumMap's iteration order changes accordingly.

Therefore:

> Enum declaration order is part of the natural ordering used by EnumMap.

---

45. Don't Confuse With `Comparator`

TreeMap allows:

```java id="v8m3q1" Comparator
to define ordering.

EnumMap doesn't need a user-supplied Comparator for its normal ordering.

It follows the enum's natural/declaration order.

---

# 46. EnumMap vs TreeMap for Enums

Suppose:

```java id="n4x7p2"
enum Priority {
```
```

```
    LOW,
    MEDIUM,
    HIGH
}
```

Both can technically represent:

```text id="k6m2r9" Priority → value
```

```
But:

```text id="p8q3x5"
EnumMap
```

is the natural specialized choice.

Why?

```text id="j5r9m1" enum-specific array-based compact fast declaration-order iteration
```

```
TreeMap is more general and uses a balanced tree.

---

# 47. EnumMap vs HashMap for Enums

If your key is definitely an enum:

```text id="x2m7q4"
EnumMap
```

is generally preferable when you want a Map specialized for that enum key type.

You get:

```text id="q8r3n6" better memory characteristics fast operations predictable enum ordering type specialization
```

```
HashMap remains useful if your design requires a general-purpose Map abstraction
or other semantics.

---
```

```
# 48. ⭐ Interview Question

### Why is EnumMap faster than HashMap?

Strong answer:

> EnumMap knows the finite set of possible enum keys and uses an array-based
representation, allowing efficient direct indexing rather than general-purpose
hash-table lookup.

---

# 49. ⭐ Interview Question

### What is the internal data structure of EnumMap?

Answer:

> It is internally array-based and uses the enum's known universe/order to
associate enum constants with array positions.

---

# 50. ⭐ Interview Question

### Can EnumMap have null keys?

No.

```text id="m7x2p8"
null key → NullPointerException
```

But null values are allowed.

---

51. ⭐ Interview Question

Does EnumMap maintain insertion order?

No.

It maintains:

> **Enum declaration order.**
```

Example:

```text id="c4r8m1" enum X { C, A, B }

```
EnumMap iteration follows:

```text id="n6x2p9"
C
A
B
```

regardless of insertion sequence.

---

52. ⭐ Interview Question

What happens if you insert enum constants in reverse order?

Example:

```java id="p8m4q2" map.put(Status.COMPLETED, "C"); map.put(Status.NEW, "N");
map.put(Status.PROCESSING, "P");

```
Iteration still follows the enum declaration order:

```text id="x3r7k5"
NEW
PROCESSING
COMPLETED
```

---

53. ⭐ Interview Question

EnumMap vs EnumSet?

Strong answer:

> EnumSet is a Set specialized for enum constants and represents membership, while EnumMap is a Map specialized for associating enum constants with values.

Think:

```text id="q5m8n2" EnumSet → enum values only

EnumMap → enum → value

```
---

54. ⭐ Interview Question

Can EnumMap store null values?

Yes.

```java id="r7x2m5"
map.put(Status.NEW, null);
```

is allowed.

---

55. ⭐ Interview Question

Is EnumMap thread-safe?

No.

You need appropriate synchronization when using it concurrently.

---

56. ⭐ Interview Question

Why is EnumMap memory efficient?

Because it can use a compact array-oriented representation based on the finite enum key space instead of general-purpose hash-table nodes and buckets.

---

57. ⭐ Coding Question

```java id="k4m8p2" enum Day { MONDAY, TUESDAY, WEDNESDAY }

EnumMap<Day, String> map = new EnumMap<>(Day.class);

map.put(Day.WEDNESDAY, "C"); map.put(Day.MONDAY, "A"); map.put(Day.TUESDAY, "B");
```

for (Day day : map.keySet()) { System.out.println(day); }

```
Output:

```text id="x8r3m6"
MONDAY
TUESDAY
WEDNESDAY
```

Insertion order is irrelevant.

---

58. ⭐ Coding Question

```java id="p2m7q9" EnumMap<Day, String> map = new EnumMap<>(Day.class);

map.put(Day.MONDAY, "A"); map.put(Day.MONDAY, "B");

System.out.println(map.size()); System.out.println(map.get(Day.MONDAY));
```

```
Output:

```text id="j6x4r1"
1
B
```

Map semantics still apply.

---

59. ⭐ Coding Question

```java id="n5k8m2" EnumMap<Day, String> map = new EnumMap<>(Day.class);

map.put(null, "X");
```

```
Result:

```text id="q7r3p9"
NullPointerException
```
```

EnumMap doesn't permit null keys.

---

60. ⭐ Coding Question

```java id="v4m9x2" EnumMap<Day, String> map = new EnumMap<>(Day.class);

map.put(Day.MONDAY, null);

System.out.println( map.containsKey(Day.MONDAY) );

System.out.println( map.get(Day.MONDAY) );
```

```
Output:

```text id="f8p3k6"
true
null
```

This demonstrates why `containsKey()` matters.

---

61. ⭐ Coding Question — EnumMap vs HashMap

Given:

```java id="r2m7x4" enum State { START, RUNNING, STOPPED }
```

```
Which is more specialized?

```java id="j5q8n1"
EnumMap<State, String>
```

rather than:

```java id="x3m9p6" HashMap<State, String>
```

```
because the key type is known to be an enum.

---
```

```
# 62. ⭐ Coding Question – State Machine

```java id="c6r2m8"
enum State {
    NEW,
    PROCESSING,
    COMPLETED,
    FAILED
}

EnumMap<State, Set<State>> transitions =
    new EnumMap<>(State.class);
```

This can naturally represent:

```text id="p8x4k1" NEW ↓ PROCESSING ├──→ COMPLETED └──→ FAILED
```

```
This is an excellent practical use of EnumMap.

---

# 63. Important Trap – EnumMap Is Not a Set

Don't confuse:

```text id="n4m7q2"
EnumMap<K,V>
```

with:

```text id="w6x3p8" EnumSet<E>
```

```
Use EnumMap when you need:

```text id="a9r2k5"
enum → value
```

Use EnumSet when you need:

```text id="j7m4x8" selected enum constants
```
```

````
---

# 64. Important Trap — EnumMap Is Not a HashMap Optimization

Don't describe EnumMap as:

> "HashMap but faster for enums."

A better answer:

> "EnumMap is a specialized Map implementation with a different, enum-aware
internal representation."

Its purpose is specialization, not simply optimization of HashMap.

---

# 65. Important Trap — EnumMap Is Not Sorted by Name

Suppose:

```java id="f5q8m2"
enum Status {
    ZEBRA,
    APPLE,
    MANGO
}
````

EnumMap order is:

```text id="p7x3r9" ZEBRA APPLE MANGO
````

not:

```text id="k2m6q8"
APPLE
MANGO
ZEBRA
```

It follows declaration order, not alphabetical order.

```

66. Important Trap — EnumMap Doesn't Use `name()`

The ordering is not based on:

```java id="r8m4p1" Status.name()
```

```
It is based on the enum's natural/declaration order.

---

# 67. Important Trap — Don't Change Enum Order Casually

Because enum declaration order influences:

```text id="m6x2q9"
ordinal()
```

and EnumMap iteration order, changing enum constant order can affect behavior.

Therefore enum ordering should be treated carefully when it has semantic significance.

---

68. EnumMap Decision Framework

```text id="q4m8x2" Need Map ↓ Are keys enums? ↓ NO ——————————————→ HashMap / LinkedHashMap / TreeMap | YES ↓ Need enum → value mapping? ↓ EnumMap
```

```
Need only enum membership?

```text id="j7p3m9"
EnumSet
```

Need insertion/access order for arbitrary keys?

```text id="x5r8k1" LinkedHashMap
```

```
Need sorted arbitrary keys?
```

```text id="n2m6q4"
TreeMap
```

---

69. Map Selection — Updated

At this point your Map mental model should be:

```text id="c8r4m1"
                                Map
                                 |
        ┌────────────────────────┼────────────────────────┐
        ↓                        ↓                        ↓
    HashMap                LinkedHashMap             TreeMap
    ↓ Fast lookup          ↓ Insertion/access order  ↓ Sorted keys
```

```
Specialized:

```text id="p5m9x2"
WeakHashMap
→ weak keys

IdentityHashMap
→ identity keys

EnumMap
→ enum keys

ConcurrentHashMap
→ concurrent access
```

🧠 Final Mental Model

Remember EnumMap with one diagram:

```text id="r7x3m8"
EnumMap | enum key | ordinal | array position | value
```

```
Example:

```text id="k4m9p2"
enum Priority {
    LOW,       // 0
    MEDIUM,    // 1
```

```
    HIGH       // 2
}
```

Conceptually:

```text id="w6q2x8" array[0] → value for LOW array[1] → value for MEDIUM array[2] → value for HIGH
```

```
Therefore:

```text id="m3r8k5"
EnumMap
  ↓
specialized
  ↓
array-based
  ↓
O(1)-style basic operations
  ↓
enum declaration-order iteration
```

🔥 Chapter Summary

Concept	Key Point
EnumMap	Map specialized for enum keys
Package	`java.util`
Key type	Enum only
Internal representation	Array-based
Hashing	Not HashMap-style hashing
Basic lookup	O(1)
Basic insertion	O(1)
Basic removal	O(1)
Iteration order	Enum declaration order
Insertion order	❌
Sorted alphabetically	❌
Null key	❌
```

Concept	Key Point
Null values	✅
Duplicate keys	❌
Duplicate values	✅
Thread-safe	❌
Memory efficiency	High for enum-key mappings
Main use	Enum → value mapping
EnumSet difference	EnumSet stores membership; EnumMap stores values

---

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] What EnumMap is
- [ ] Why EnumMap exists
- [ ] Why enum keys allow specialization
- [ ] Array-based internal representation
- [ ] Enum ordinal/declaration position concept
- [ ] Why basic operations are O(1)
- [ ] Why EnumMap can be memory efficient
- [ ] Enum declaration-order iteration
- [ ] Why insertion order doesn't matter
- [ ] Why it doesn't sort alphabetically
- [ ] Why null keys aren't allowed
- [ ] Why null values are allowed
- [ ] `EnumMap` vs `HashMap`
- [ ] `EnumMap` vs `LinkedHashMap`
- [ ] `EnumMap` vs `TreeMap`
- [ ] `EnumMap` vs `EnumSet`
- [ ] State-machine use case
- [ ] Strategy lookup use case
- [ ] Configuration lookup use case
- [ ] Why enum ordinal shouldn't be used as a persistent ID
- [ ] Why EnumMap isn't thread-safe
- [ ] How `containsKey()` differs from `get()` when null values are possible

🚀 Next Chapter

Chapter 21 — ConcurrentHashMap (Complete Deep Dive)

We'll cover the **most important concurrent Map implementation for interviews**:

- Why HashMap is unsafe in multithreading
- Why `Hashtable` is outdated
- ConcurrentHashMap architecture
- Java 7 segmented design
- Java 8+ design
- CAS
- Volatile reads
- Locking at bucket/bin level
- Concurrent reads
- Concurrent writes
- Why it doesn't lock the entire Map
- Hash collisions
- Tree bins
- `size()` and weakly consistent behavior
- `compute()`
- `computeIfAbsent()`
- `putIfAbsent()`
- Atomic compound operations
- Why null keys/values aren't allowed
- Fail-safe vs weakly consistent iterators
- ConcurrentHashMap vs HashMap
- ConcurrentHashMap vs Hashtable
- ConcurrentHashMap vs synchronizedMap
- Real-world caching/counter patterns
- 25+ senior-level interview questions